SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1707

COURSE OUTCOME COVERED

CO5: Design AI-based systems using Py Spark, RDD operations, and intelligent agent architectures, using

scalable systems and integrate components in distributed AI applications. (BTL 5)

Assessment Tool Weightage: 50%

Duration : 1 Hour

QUESTIONS: 5 Total Marks:50

Annexure A

Design Task

Code of Conduct:

I J O JOANNA DIVINE(192572003) (Name / Reg No) certify that this submission is my original work and that I have adhered

to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

SIGNATURE OF THE STUDENT:JOANNA

ANNEXURE A

DESIGN TASK ASSIGNMENT

Distributed AI Systems Using PySpark and Intelligent Agents

---

1. Hybrid AI Recommendation Engine Design

1.1 Aim

To design a hybrid AI-based recommendation system that combines content-based filtering and collaborative filtering using PySpark RDDs to provide personalized recommendations for large numbers of users.

1.2 Problem Statement

Modern applications generate millions of user-item interactions such as product purchases, movie ratings, music plays, and online searches. Processing this large-scale data using traditional systems can be slow. A distributed recommendation system using PySpark can process user-item data efficiently and generate personalized recommendations.

1.3 System Architecture

The proposed system consists of:

1. Data Collection Layer – Collects user profiles, item information, ratings, clicks, and purchase history.
2. PySpark Processing Layer – Stores and processes large datasets using RDDs.
3. Content-Based Filtering Module – Recommends items similar to those previously preferred by the user.
4. Collaborative Filtering Module – Finds similarities between users and items based on interaction patterns.
5. Hybrid Recommendation Module – Combines both recommendation techniques.
6. Intelligent Agent Layer – Monitors user behavior and continuously updates recommendations.
7. Recommendation Output Layer – Displays personalized recommendations.

1.4 Use of PySpark RDDs

RDD operations can be used as follows:

- map() – Transform user-item records.
- filter() – Remove invalid or unwanted interactions.
- flatMap() – Generate multiple recommendation features.

- reduceByKey() – Aggregate ratings or interaction scores.
- groupByKey() – Group items based on users.
- join() – Combine user and item information.

For example, millions of user-item interactions can be divided into partitions and processed simultaneously across multiple worker nodes.

1.5 Hybrid Recommendation Approach

The system generates two scores:

Content Score: Based on similarity between item features.

Collaborative Score: Based on preferences of similar users.

The final recommendation score can be calculated as:

Final Score = α(Content Score) + β(Collaborative Score)

where α and β are weighting factors.

1.6 Role of Intelligent Agents

Intelligent agents continuously observe:

- User searches
- Clicks
- Purchases
- Ratings
- Time spent on items
- Likes and dislikes

The agent updates the user's preference profile and changes recommendations dynamically.

For example, if a user suddenly starts watching sports videos, the agent can increase the priority of sports-related content.

1.7 Advantages

- Handles millions of interactions.
- Provides personalized recommendations.
- Supports distributed processing.
- Adapts to changing user preferences.
- Reduces processing time.

1.8 Expected Outcome

The system produces accurate and continuously updated recommendations while efficiently processing large-scale user-item interaction data.

1.9 Conclusion

A hybrid recommendation engine combining content-based and collaborative filtering provides better personalization than using either technique alone. PySpark RDDs enable scalable distributed processing, while intelligent agents allow the system to adapt recommendations according to real-time user behavior.

---

2. Smart Disaster Detection System

2.1 Aim

To design a distributed AI system using PySpark RDDs for detecting natural disasters such as floods and earthquakes by combining satellite images, sensor readings, and social-media information.

2.2 Problem Statement

Natural disasters generate huge amounts of information from multiple sources. Processing this information quickly is important for early detection and emergency response.

2.3 System Architecture

The proposed system contains:

1. Data Sources
    - Satellite images
    - IoT sensors
    - Weather stations
    - Social media
    - Emergency reports
2. Data Ingestion Layer
3. PySpark Distributed Processing Layer
4. Data Fusion Module
5. AI Disaster Detection Module
6. Intelligent Agent Coordination Layer
7. Early Warning System
8. Emergency Response Module

2.4 Data Processing Using RDDs

PySpark RDDs process large volumes of disaster-related information.

Typical operations include:

- map() for transforming sensor records.
- filter() for removing noisy readings.
- reduceByKey() for aggregating sensor measurements.
- join() for combining information from different sources.
- groupByKey() for grouping data by geographical region.

Distributed processing allows different geographical areas to be processed simultaneously.

2.5 Data Fusion

The system combines multiple sources to improve detection accuracy.

For example:

Satellite Data + Sensor Data + Social Media Data → Disaster Confidence Score

If water-level sensors report rising water, satellite images show increased water coverage, and social media contains flood-related reports, the system can generate a high flood-risk score.

2.6 Role of Intelligent Agents

Different agents can perform different tasks:

- Monitoring Agent – Monitors sensor and satellite information.
- Detection Agent – Identifies possible disasters.
- Verification Agent – Cross-checks information from different sources.
- Warning Agent – Generates alerts.
- Response Agent – Suggests evacuation and emergency actions.

The agents communicate with one another to make faster decisions.

2.7 Advantages

- Faster disaster detection.
- Large-scale data processing.
- Multi-source information fusion.
- Reduced false alarms.
- Real-time warning generation.

2.8 Expected Outcome

The system detects possible disasters at an early stage and provides timely warnings and response recommendations.

2.9 Conclusion

A distributed disaster detection system using PySpark can efficiently process data from multiple sources. Intelligent agents improve coordination between detection, verification, warning, and response activities, helping authorities respond more quickly.

---

3. AI-Based Fraud Detection Framework

3.1 Aim

To design a scalable AI-based fraud detection framework using PySpark RDDs to analyze large volumes of financial transactions and identify suspicious activities.

3.2 Problem Statement

Financial institutions process millions of transactions every day. Fraudulent transactions may have unusual characteristics such as abnormal transaction amounts, unusual locations, or unexpected transaction frequencies.

3.3 System Architecture

The system consists of:

1. Transaction Data Collection
2. Data Cleaning
3. PySpark RDD Processing
4. Feature Extraction
5. Distributed Anomaly Detection
6. Intelligent Fraud Detection Agents
7. Alert Generation
8. Continuous Model Improvement

3.4 RDD-Based Processing

Transaction records can contain:

- Transaction ID
- Customer ID
- Amount
- Location

- Timestamp
- Merchant category
- Payment method

RDD operations can process these records:

- filter() – Remove invalid transactions.
- map() – Extract transaction features.
- reduceByKey() – Calculate transaction statistics.
- groupByKey() – Group transactions by customer.
- join() – Combine customer and transaction information.

3.5 Anomaly Detection

The system establishes normal transaction behavior for each customer.

A transaction can be considered suspicious when it significantly differs from the customer's normal behavior.

Example:

A customer normally performs transactions below ₹5,000 in one location. Suddenly, several high-value transactions occur from different locations within a short period. The system assigns a high anomaly score.

3.6 Intelligent Agents

Multiple intelligent agents collaborate:

Transaction Monitoring Agent

Continuously monitors transactions.

Anomaly Detection Agent

Identifies unusual transaction patterns.

Verification Agent

Checks suspicious transactions against historical behavior.

Alert Agent

Generates alerts for high-risk transactions.

Learning Agent

Uses confirmed fraud cases to improve future detection.

3.7 Advantages

- Scalable transaction processing.
- Faster fraud detection.
- Distributed anomaly analysis.
- Continuous learning.
- Reduced financial losses.

3.8 Expected Outcome

The system identifies suspicious transactions quickly and generates alerts while continuously improving fraud detection accuracy.

3.9 Conclusion

PySpark RDDs provide a scalable framework for processing large financial datasets. Intelligent agents enhance the system by continuously monitoring transactions, identifying anomalies, coordinating verification, and learning from new fraud patterns.

---

4. Autonomous Supply Chain Optimization System

4.1 Aim

To design a distributed AI-based supply chain optimization system using PySpark and intelligent agents to improve inventory management, logistics, demand forecasting, and cost efficiency.

4.2 Problem Statement

Supply chains generate large amounts of information from warehouses, suppliers, transportation systems, sales platforms, and customers. Efficient processing of this information is required to reduce inventory costs and delivery delays.

4.3 System Architecture

The proposed system includes:

1. Sales Data Collection
2. Inventory Data Collection
3. Supplier Information
4. Transportation Data
5. PySpark RDD Processing
6. Demand Forecasting
7. Inventory Optimization
8. Route Optimization

9. Intelligent Agent Coordination
10. Decision-Making Module

4.4 RDD Transformations

RDDs can process large-scale supply chain information using:

- map() – Transform sales and inventory records.
- filter() – Remove invalid records.
- reduceByKey() – Calculate total demand.
- groupByKey() – Group products by warehouse.
- join() – Combine inventory and demand information.

RDD partitions allow different warehouses or product groups to be processed in parallel.

4.5 Demand Forecasting

Historical sales data can be analyzed to predict future demand.

For example:

Historical Sales → Feature Extraction → AI Forecasting → Expected Demand

The system can use the prediction to determine how much inventory should be maintained.

4.6 Intelligent Agents

Different agents can work independently but coordinate their decisions.

- Inventory Agent – Monitors stock levels.
- Demand Agent – Forecasts future demand.
- Logistics Agent – Selects efficient transportation routes.
- Supplier Agent – Monitors supplier performance.
- Cost Agent – Attempts to minimize operational costs.
- Coordination Agent – Coordinates decisions between all agents.

4.7 Decision-Making

If inventory is low and predicted demand is high, the inventory agent can recommend replenishment.

If transportation costs increase on one route, the logistics agent can identify an alternative route.

4.8 Advantages

- Reduced inventory cost.

- Improved demand forecasting.
- Faster decision-making.
- Better logistics management.
- Efficient resource utilization.
- Distributed processing of large datasets.

4.9 Expected Outcome

The system automatically recommends inventory and logistics decisions that reduce operational costs while maintaining product availability.

4.10 Conclusion

An autonomous supply chain system using PySpark and intelligent agents can process large-scale operational data and make decentralized decisions. Coordination among agents improves efficiency, reduces costs, and increases supply chain responsiveness.

---

5. Personalized Learning Analytics Platform

5.1 Aim

To design an AI-driven educational platform using PySpark RDDs and intelligent agents to analyze student learning behavior and provide personalized educational content.

5.2 Problem Statement

Online education platforms generate large amounts of student data, including quiz scores, assignment submissions, video watching behavior, attendance, and learning time. Analyzing this information can help provide personalized learning.

5.3 System Architecture

The proposed system consists of:

1. Student Data Collection
2. Learning Activity Database
3. PySpark Distributed Processing
4. Student Performance Analysis
5. Learning Behavior Analysis
6. AI Recommendation Module
7. Intelligent Learning Agents
8. Personalized Content Delivery

9. Real-Time Feedback System

5.4 Student Data

The system can analyze:

- Quiz scores
- Assignment marks
- Attendance
- Video completion
- Time spent learning
- Number of attempts
- Topic-wise performance
- Learning frequency

5.5 RDD Processing

RDD operations can process student records:

- map() – Extract learning features.
- filter() – Remove incomplete records.
- groupByKey() – Group activities by student.
- reduceByKey() – Calculate average scores.
- join() – Combine performance and course information.

For thousands of students, distributed processing allows different student groups to be analyzed in parallel.

5.6 Personalized Learning

The system identifies each student's strengths and weaknesses.

For example:

Student Performance → Weak Topic Detection → Content Recommendation → Practice → Feedback

If a student performs poorly in database normalization, the system may recommend additional tutorials, examples, and practice questions related to normalization.

5.7 Intelligent Agents

Student Monitoring Agent

Tracks learning behavior.

Performance Agent

Analyzes marks and identifies weak areas.

Recommendation Agent

Recommends suitable learning resources.

Feedback Agent

Provides immediate feedback.

Adaptation Agent

Changes the difficulty level according to student performance.

5.8 Real-Time Adaptation

The intelligent agent continuously receives new student information.

If performance improves, the system can provide advanced questions. If performance decreases, it can recommend simpler explanations and additional practice.

5.9 Advantages

- Personalized learning.
- Real-time feedback.
- Scalable student analytics.
- Early identification of weak areas.
- Adaptive content delivery.
- Improved learning experience.

5.10 Expected Outcome

The platform provides personalized educational content to thousands of learners while continuously adapting to their performance and learning behavior.

5.11 Conclusion

A Personalized Learning Analytics Platform using PySpark RDDs can efficiently analyze large-scale educational data. Intelligent agents enable continuous monitoring, personalized recommendations, and adaptive learning, making education more effective and student-centered.

Overall Conclusion

The five proposed systems demonstrate how PySpark RDDs and intelligent agents can be combined to build scalable and adaptive AI applications. PySpark provides distributed

processing for large datasets, while intelligent agents provide autonomous monitoring, decision-making, coordination, and adaptation.

The combination of distributed computing and intelligent agents is suitable for real-world applications such as recommendation systems, disaster management, fraud detection, supply chain optimization, and personalized education.

RUBRICS FOR CALCULATIONS:

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed

Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation